1) Explain why REST APIs are preferred for mobile and frontend-heavy apps

REST APIs are preferred because they provide fast, simple, and flexible communication between the frontend (mobile/web app) and backend (server/database).

Main Reasons

1. Platform Independent

REST APIs work with any platform or language.

Example:

- Backend → Python, Java, PHP
- Frontend → React, Angular
- Mobile → Android, iOS

All can communicate using REST APIs.

2. Uses JSON Format

REST APIs mostly use JSON data which is lightweight and easy to understand.

Example:

```
{
   "name": "Dr Raj",
   "specialization": "Cardiologist"
}
```

JSON loads faster in mobile apps and websites.

3. Faster Performance

REST APIs send only required data, so applications become faster and use less internet data.

This is very useful for mobile applications.

4. Easy Frontend Development

Frontend developers can use APIs directly without depending on backend page design.

Example:

```
GET /doctors/
POST /doctors/
```

This makes development faster.

5. Supports CRUD Operations

REST APIs use HTTP methods:

Method	Work
GET	Read data
POST	Insert data
PUT	Update data
DELETE	Delete data

6. Easy Testing

REST APIs can be tested easily using Postman or browser.

Conclusion

REST APIs are preferred for mobile and frontend-heavy apps because they are fast, lightweight, scalable, easy to use, and support communication between different platforms using JSON and HTTP methods.

2) Explain how serializers act as a validation layer and how to implement custom field-level validation.

In Django REST Framework, serializers convert complex model data into JSON format and also validate the incoming data before saving it into the database. Therefore, serializers work as a validation layer between the client request and the database.

When a user sends data through an API request, the serializer checks whether the data is correct according to the defined rules. If the data is invalid, the serializer returns error messages instead of saving incorrect data.

For example, serializers can validate:

- Required fields
- Maximum length
- Data types
- Unique values
- Custom conditions

Custom Field-Level Validation

Custom field-level validation is implemented using methods starting with validate_<fieldname> inside the serializer class.

Example:

```
from rest_framework import serializers
from myapp.models import Doctor

class Doctorser(serializers.ModelSerializer):

    class Meta:
        model = Doctor
        fields = '__all__'

    # Custom Validation
    def validate_name(self, value):

        if len(value) < 3:
            raise serializers.ValidationError(
                "Name must contain at least 3 characters."
            )

        return value
```

In this example, the serializer checks whether the doctor name contains at least 3 characters. If the condition fails, an error message is returned and the data is not saved.

Thus, serializers improve data security, maintain data integrity, and prevent invalid data from entering the database.

3) Explain the importance of using appropriate HTTP status codes (201 Created vs 200 OK) for API outcomes.

HTTP status codes are important in REST APIs because they inform the client whether the API request was successful or failed. Proper status codes help frontend developers, mobile applications, and API users understand the result of the request correctly.

Using correct status codes improves communication between the client and server, makes debugging easier, and follows REST API standards.

200 OK

200 OK is used when a request is successfully completed.

It is commonly used for:

- GET requests
- PUT requests
- DELETE requests

Example:

```
return Response(
    {"message": "Data Updated Successfully"},
    status=status.HTTP_200_OK
)
```

This means the requested operation was successful.

201 Created

201 Created is used when new data is successfully created in the database.

It is mainly used for POST requests.

Example:

```
return Response(
    {
        "data": ser.data,
        "message": "Data Inserted Successfully"
    },
    status=status.HTTP_201_CREATED
)
```

This indicates that a new resource has been created successfully.

Difference Between 200 OK and 201 Created

Status Code	Meaning	Common Use
200 OK	Request successful	GET, PUT, DELETE
201 Created	New resource created	POST

Conclusion

Using appropriate HTTP status codes is important because they provide clear information about API outcomes, improve client-server communication, help in debugging, and follow REST API standards. 200 OK indicates successful operations, while 201 Created specifically indicates successful resource creation.

4) Explain why pagination is required when listing doctors and how it impacts database performance.

Pagination is used in REST APIs to divide large amounts of data into smaller and manageable pages. When listing doctors, there may be hundreds or thousands of records in the database. Returning all records at once can slow down the API and increase server load.

Pagination helps in sending only a limited number of records per request, which improves application performance and user experience.

For example, instead of returning 10,000 doctor records together, the API can return only 10 or 20 records per page.

Example:

GET /doctors/?limit=10&offset=0

Impact on Database Performance

Pagination improves database performance in several ways:

- Reduces the amount of data fetched from the database
- Lowers server memory usage
- Decreases response time
- Reduces network bandwidth usage
- Improves API speed and scalability

Without pagination, large queries can make the application slow and increase database load. Pagination ensures efficient data handling, especially in mobile and frontend-heavy applications.

Conclusion

Pagination is important because it manages large datasets efficiently by returning limited records per request. It improves database performance, reduces server load, increases API speed, and provides a better user experience.

5) Explain the benefits of ViewSets over APIView for rapid CRUD development.

Benefits of ViewSets Over APIView for Rapid CRUD Development

In Django REST Framework, ViewSets are preferred for rapid CRUD development because they reduce code duplication and automatically provide standard API operations such as Create, Read, Update, and Delete.

Using APIView requires writing separate methods like `get()`, `post()`, `put()`, and `delete()` manually. This increases coding effort and development time.

On the other hand, ViewSets provide all CRUD operations automatically with minimal code.

Benefits of ViewSets

1. Less Code

ViewSets reduce boilerplate code because CRUD operations are already built-in.

Example:

```
class DoctorViewSet(viewsets.ModelViewSet):
    queryset = Doctor.objects.all()
    serializer_class = Doctorser
```

With only a few lines of code, all CRUD APIs are created automatically.

2. Faster Development

Developers can create APIs quickly without manually defining every HTTP method.

This is useful for rapid application development.

3. Automatic CRUD Operations

ViewSets automatically support:

- GET
- POST
- PUT
- DELETE

No need to write separate methods manually.

4. Easy Router Integration

ViewSets work easily with routers like `DefaultRouter`.

Example:

```
router.register(r'doctors', DoctorViewSet)
```

This automatically generates API URLs.

5. Cleaner and Maintainable Code

ViewSets make the project cleaner, shorter, and easier to maintain compared to APIView.

Conclusion

ViewSets are better for rapid CRUD development because they reduce code duplication, provide automatic CRUD operations, support router integration, speed up development, and make the code more maintainable compared to APIView.

6) Explain the use of Atomic Transactions (transaction.atomic) when creating related doctor records to ensure data integrity.

Use of Atomic Transactions (transaction.atomic) for Data Integrity

In Django REST Framework, `transaction.atomic()` is used to ensure database consistency and data integrity during multiple database operations.

When creating related doctor records, there may be several database actions happening together. For example:

- Creating doctor profile
- Saving hospital details
- Saving appointment records

If one operation fails and others are already saved, the database may contain incomplete or incorrect data.

`transaction.atomic()` solves this problem by treating all database operations as a single transaction. If any operation fails, all changes are rolled back automatically.

Benefits of transaction.atomic()

- Prevents partial data saving

- Maintains database consistency
- Ensures data integrity
- Automatically rolls back failed operations
- Makes database operations safer

Example

```
from django.db import transaction

def perform_create(self, serializer):

    with transaction.atomic():

        serializer.save()
```

In this example, if an error occurs while saving the doctor record, the transaction is cancelled and no incomplete data is stored in the database.

Conclusion

transaction.atomic() is important because it ensures that all related database operations are completed successfully together. If any operation fails, all changes are rolled back, preventing partial or corrupt data entries and maintaining data integrity.